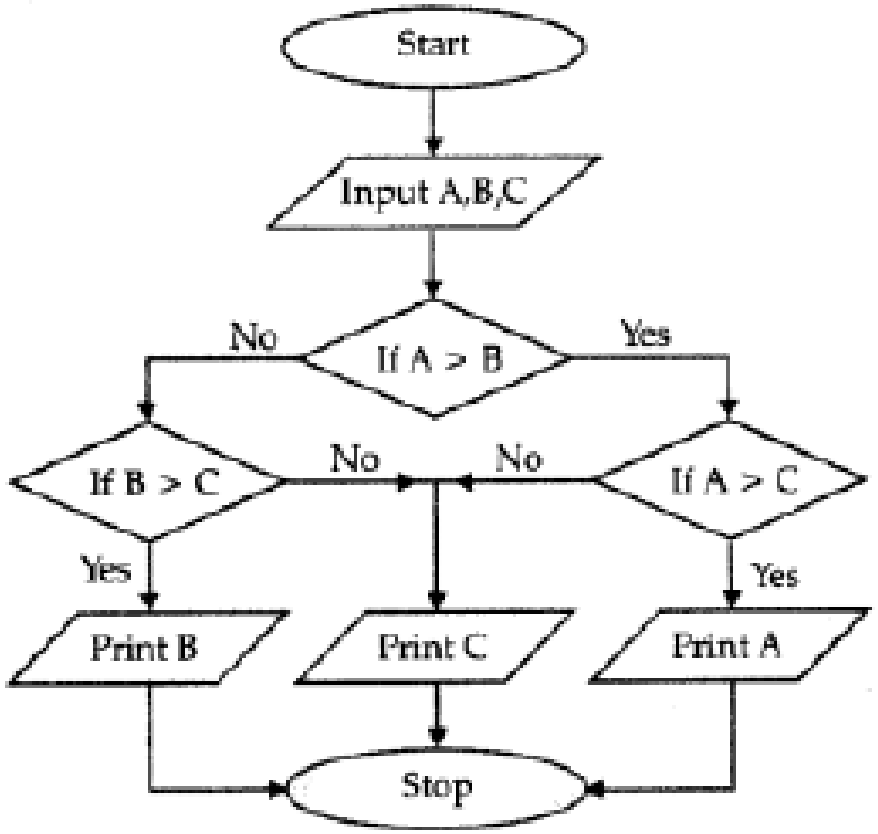
Data Structures and Algorithms
Lecture 1- Introduction & Arrays

Prof. Noel Fernando/Professor
Dept. Information Systems Engineering
UCSC

Question :Find the maximum from 3 variables (A,B,C)

Answer :Find the maximum from 3 variables (A,B,C)



Solution

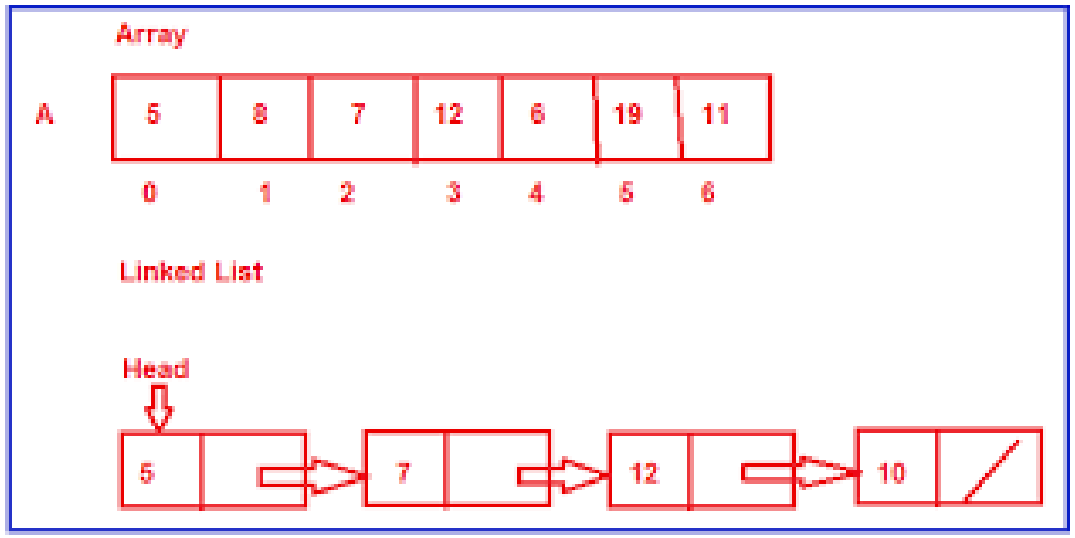
Algorithm:

1. Start
2. Input A,B,C
3. If (A>B) and (A>C) then print "A is greater".
   Else if (B>A) and (B>C) then print "B is greater".
   Else print "C is greater".
4. Stop

What is a data structure?

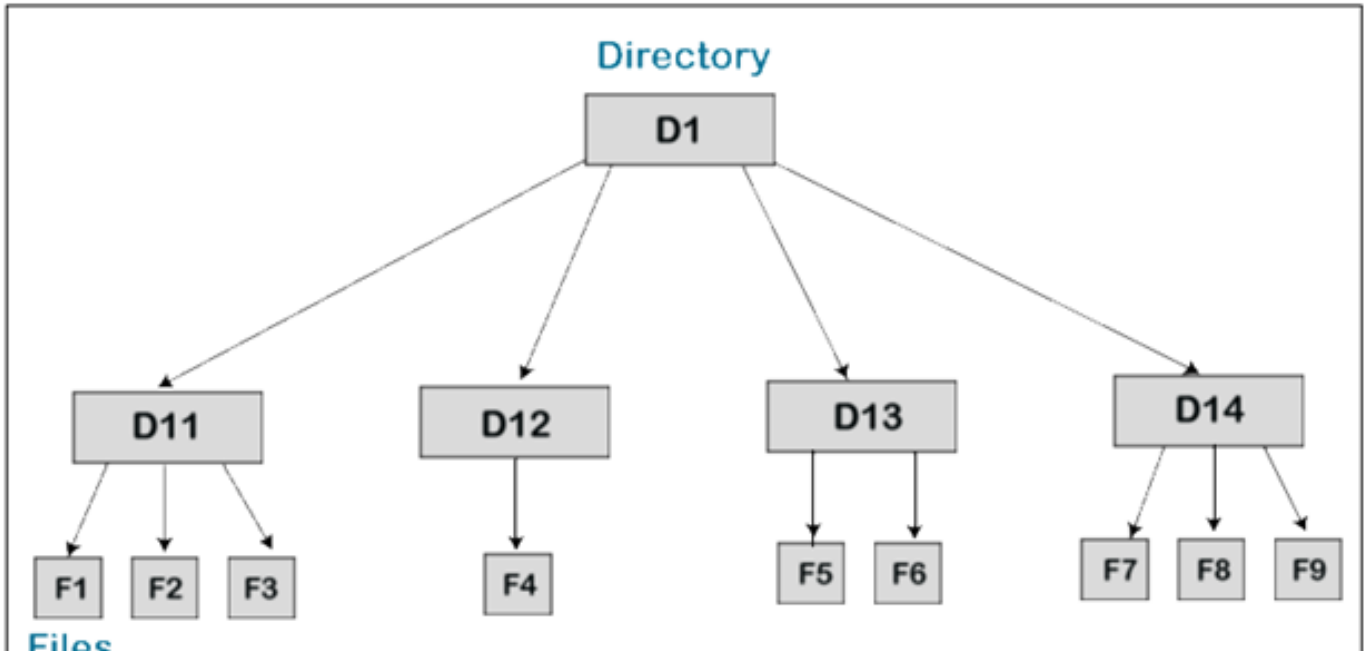
What is a data structure?

- Data Structure can be defined as the group of **data elements** which provides an **efficient** way of **storing and organizing data** in the computer so that it can be **used efficiently**.
- Some examples of Data Structures are arrays, Linked List, Stack, Queue, etc.

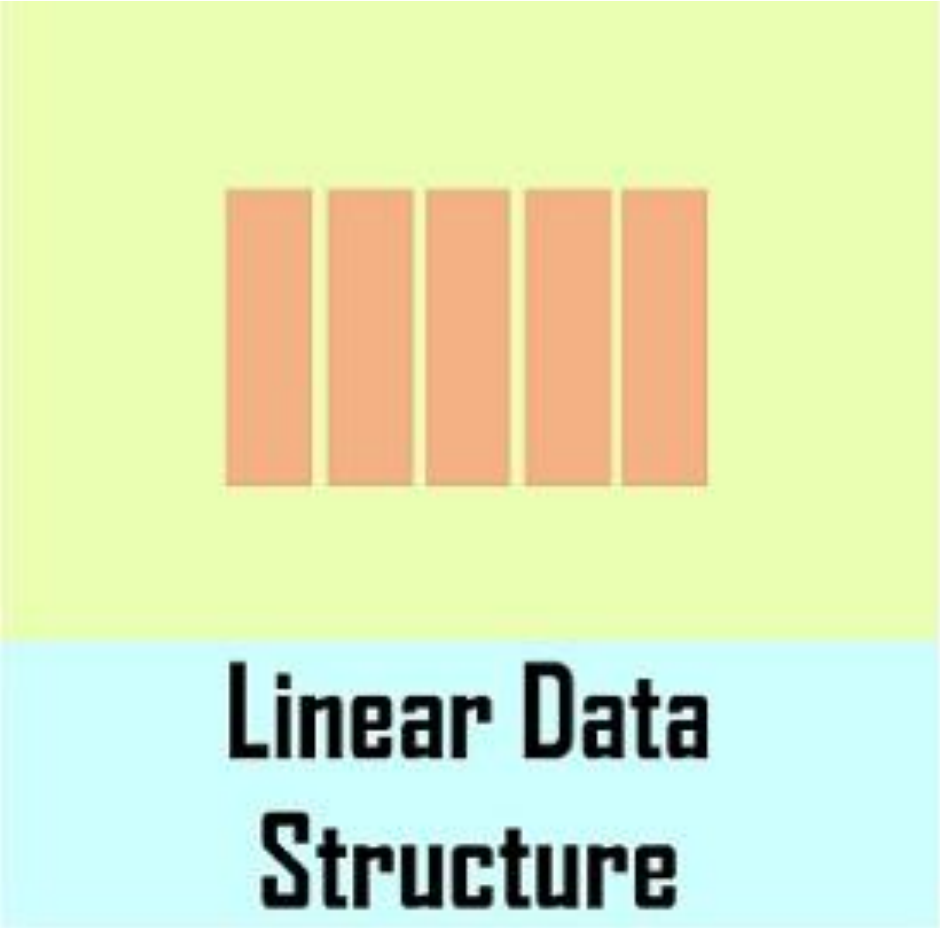


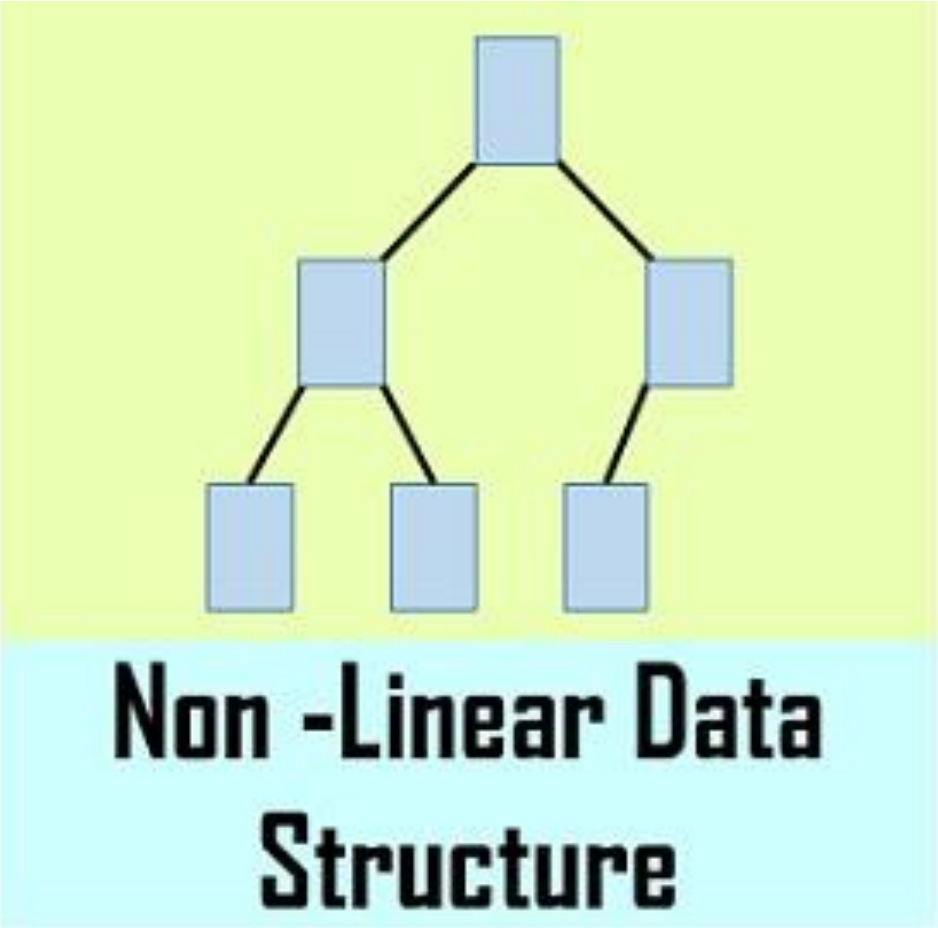
What is a data structure?

- Data Structures are widely used in almost every aspect of Computer Science
- i.e. Operating System, Compiler Design, Artificial intelligence, Graphics and many more.



Different types of Data Structures





Examples of data structures

- Arrays
- Stacks
- Queues
- Linked lists
- Trees
- Graphs
- Priority Queues

Categories of Data Structure

- Data structures can be subdivided into two major types.
- Linear Data structure
- Non-Linear Data structure

- Linear Data structure : A data structure is said to be linear if its elements combine to form any specific order.
- E.g. Arrays and Linked List.

- Non-Linear Data structure : This structure mainly represents data with a hierarchical relationship between different elements.
- E.g. Trees and Graphs

Properties of Linear and Non-linear Data Structures

BASIS FOR COMPARISON	LINEAR DATA STRUCTURE	NON-LINEAR DATA STRUCTURE
Basic	The data items are arranged in an orderly manner where the elements are attached adjacently.	It arranges the data in a sorted order and there exists a relationship between the data elements.
Traversing of the data	The data elements can be accessed in one time (single run).	Traversing of data elements in one go is not possible.
Ease of implementation	Simpler	Complex
Levels involved	Single level	Multiple level
Examples	Array, queue, stack, linked list, etc.	Tree and graph.
Memory utilization	Ineffective	Effective

Arrays

- **Variable:** Placeholder/Structure to store a basic unit of data.
- **Example:**
- One integer variable stores single integer data.
- One float variable stores single float data.

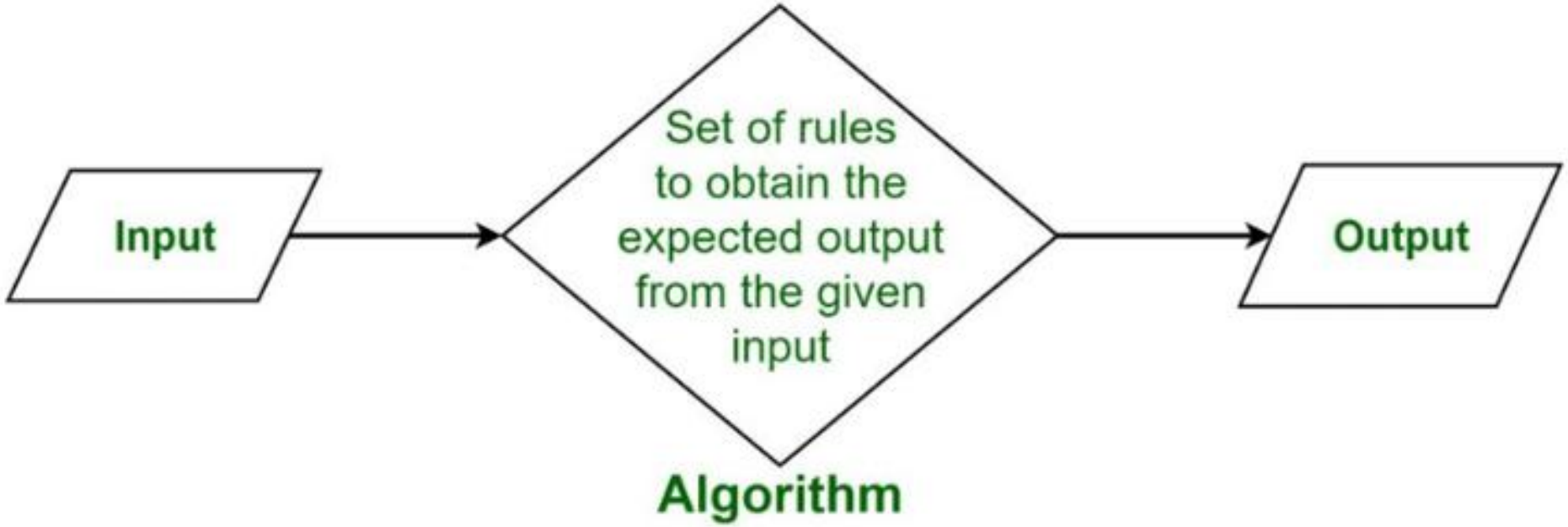
40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- **Array Indices**

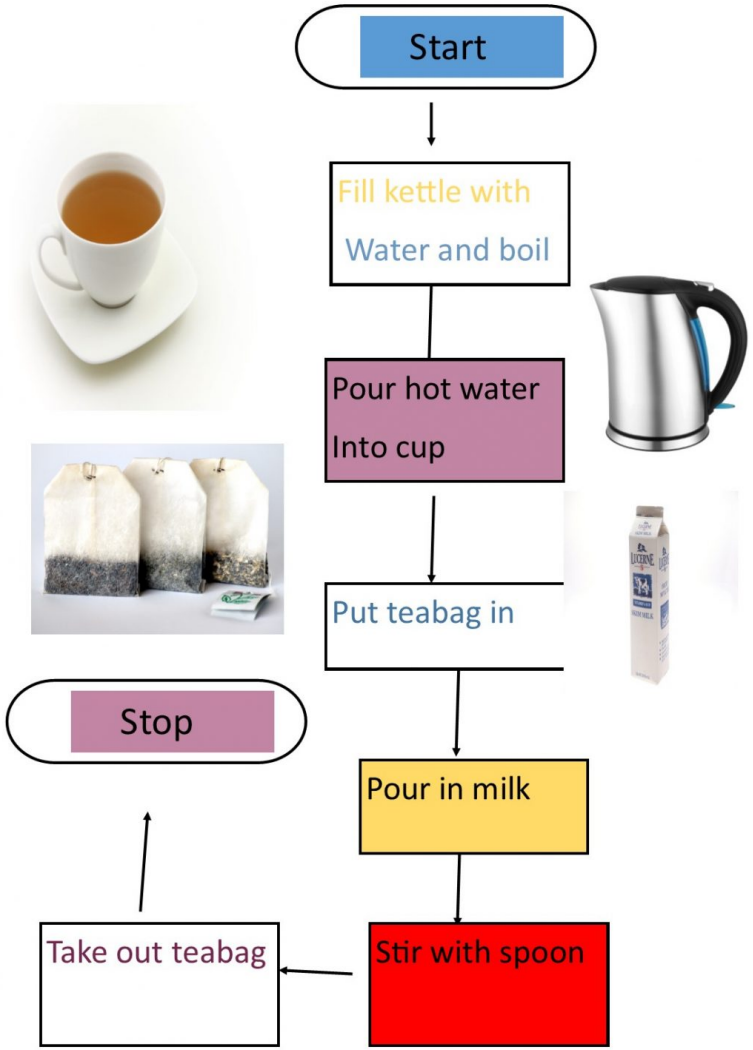
Array Length = 9
First Index = 0
Last Index = 8

What is an Algorithm?

What is an Algorithm?

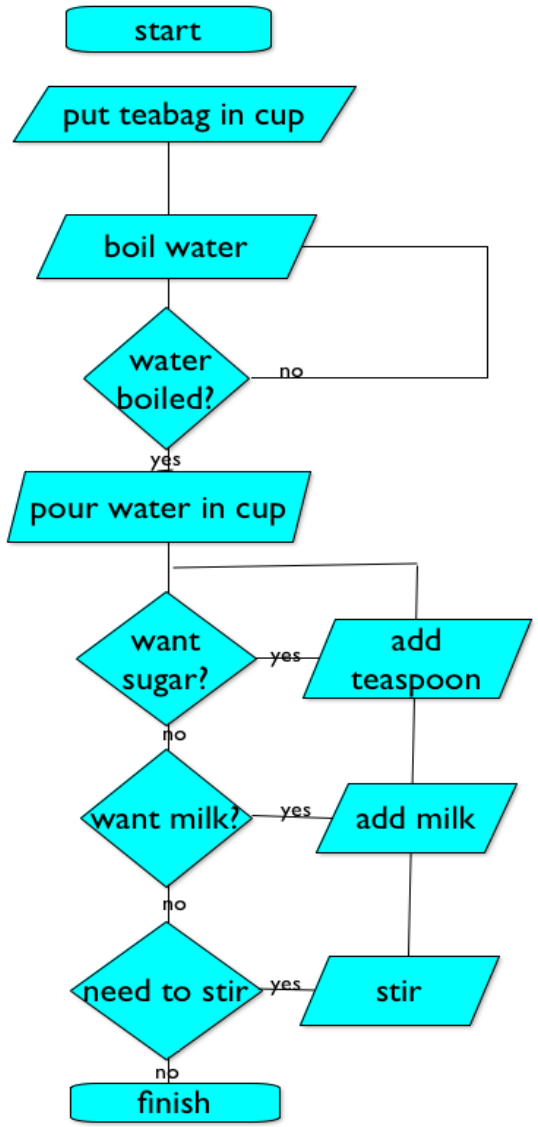


An Algorithm - For Making A Cup Of Tea



An Algorithm For Making A Cup Of Tea

Flowchart



Pseudocode

```
PROGRAM make tea

put teabag in cup

WHILE (water not boiled)
    boil water
    ENDWHILE
pour water in cup

WHILE (sugar needed)
    add sugar
ENDWHILE

WHILE (milk needed)
    add milk
ENDWHILE

WHILE (need to stir)
    stir tea
ENDWHILE

Server and finish
```

Qualities of a Good Algorithm

- Input and output should be defined precisely.
- Each step in the algorithm should be clear and unambiguous.
- Algorithms should be most effective among many different ways to solve a problem.
- An algorithm shouldn't include computer code. Instead, the algorithm should be written in such a way that it can be used in different programming languages.

Algorithm

- Algorithm is required to be:
- Well defined – it is clearly and unambiguously specified
- Effective : its steps are executable
- Finite : it terminates after a limited number of steps

Must produce a correct solution 100% of the time

Question

- In a computer, where is the value assigned to a variable stored? And where exactly in memory is it located?

Arrays

- **Question: In a computer, where is that value assigned to a variable stored?**
- Answer: In memory, possibly Random-Access Memory (RAM)

- **Question : But where exactly in memory?**
- Answer: On some physical address in memory because,

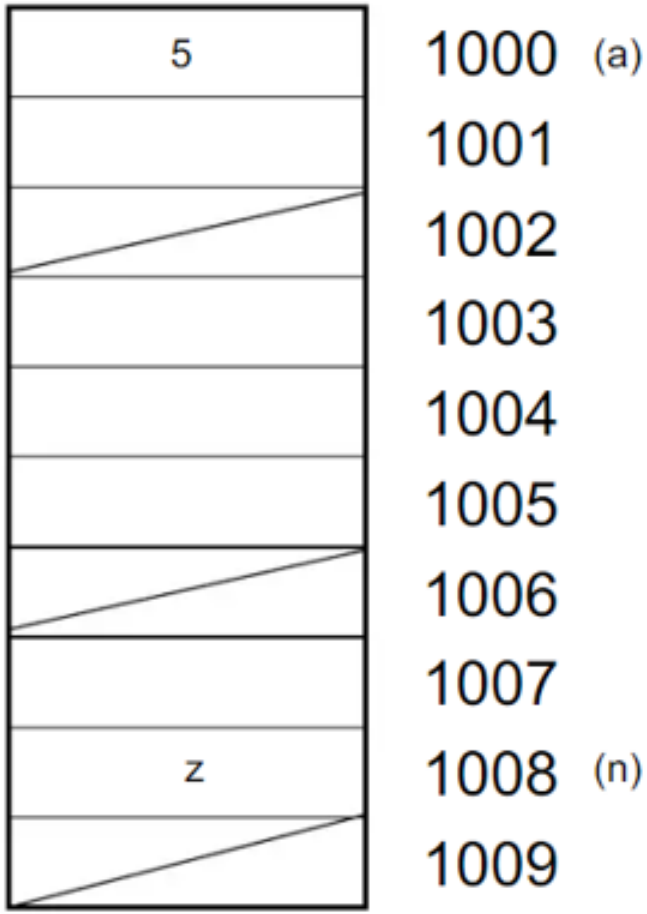
  Memory is divided in physical locations and,

  Each location has a particular address associated with it.

Variable

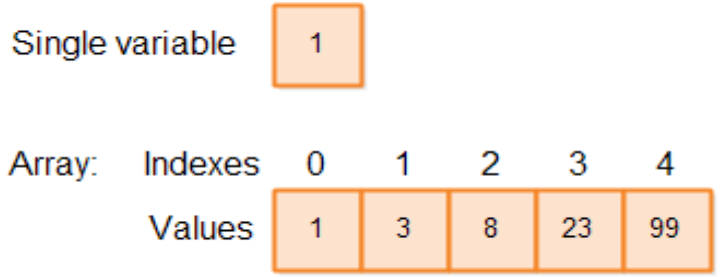
int a = 5;

char n = ‘z’;

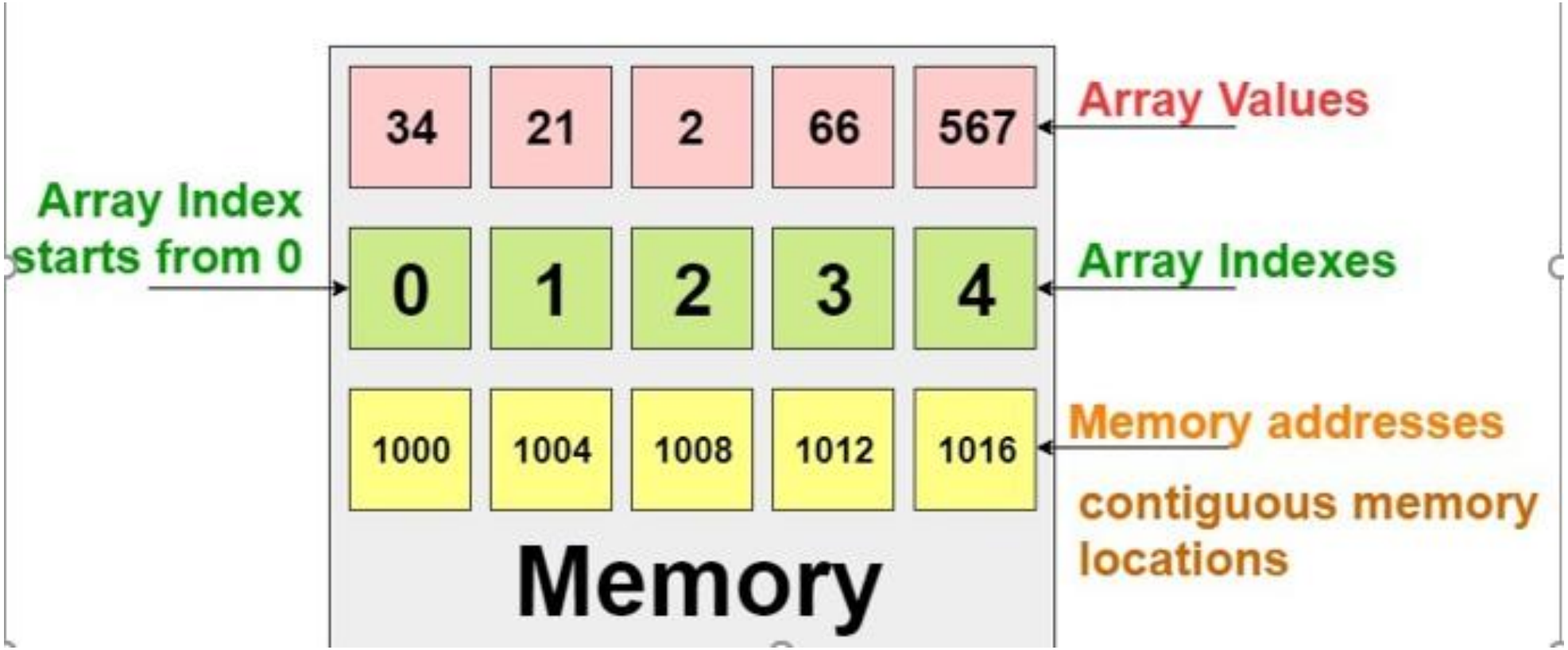


Array

- Variable: –Sometimes in our program, we need to store multiple data of same type
- Example:
- index numbers of 10 students.
- Marks of DSA of 10 students.



Arrays

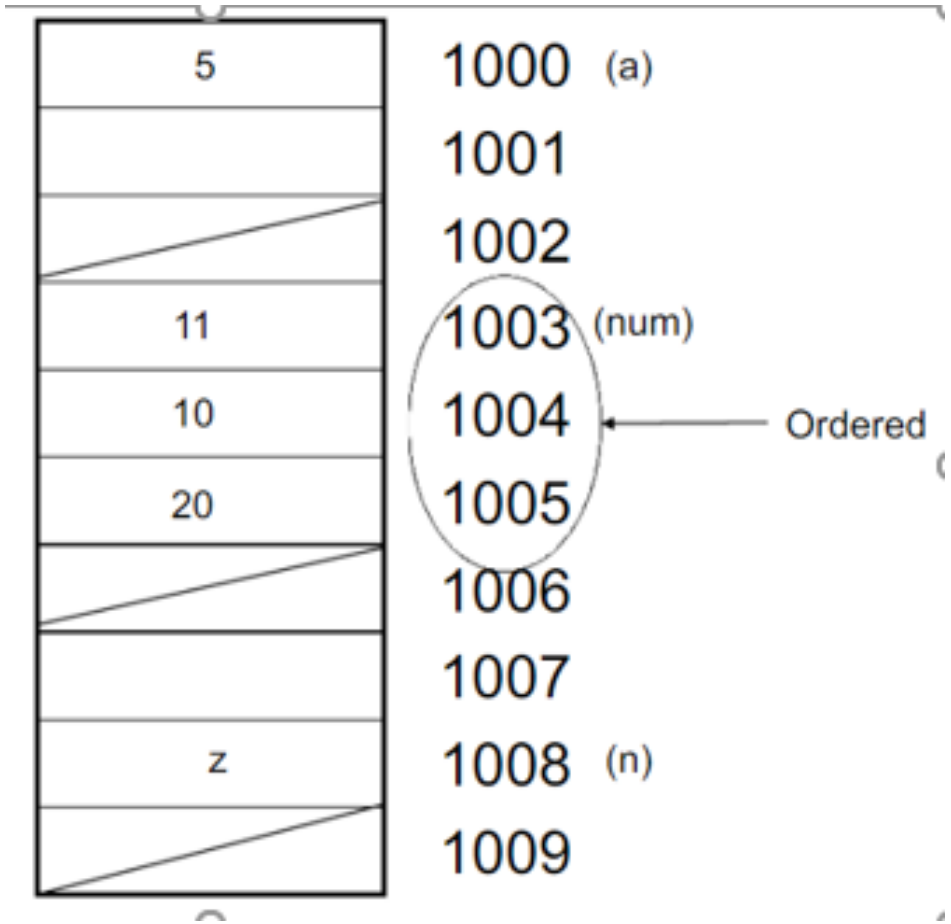

34
21
2
66
567
Array Values
Array Index
starts from 0
0
1
2
3
4
Array Indexes
1000
1004
1008
1012
1016
Memory addresses
contiguous memory
locations
Memory

Array Data Structures

- Array is a container which can hold a fix number of items, and these items should be of the same type.
- Most of the data structures make use of arrays to implement their algorithms.
- Following are the important terms to understand the concept of Array.
- **Element** – Each item stored in an array is called an element.
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.

Array representation using C

- `int a = 5;`
- `Int num []= {11, 10, 20};`
- `char myLetter = ‘z';`



Array Representation

- Arrays can be declared in various ways in different languages.
- For illustration, let's take C array declaration.
- E.g.
- Int num = [10];
- Num[0]=20
- Num[1]=30 …….

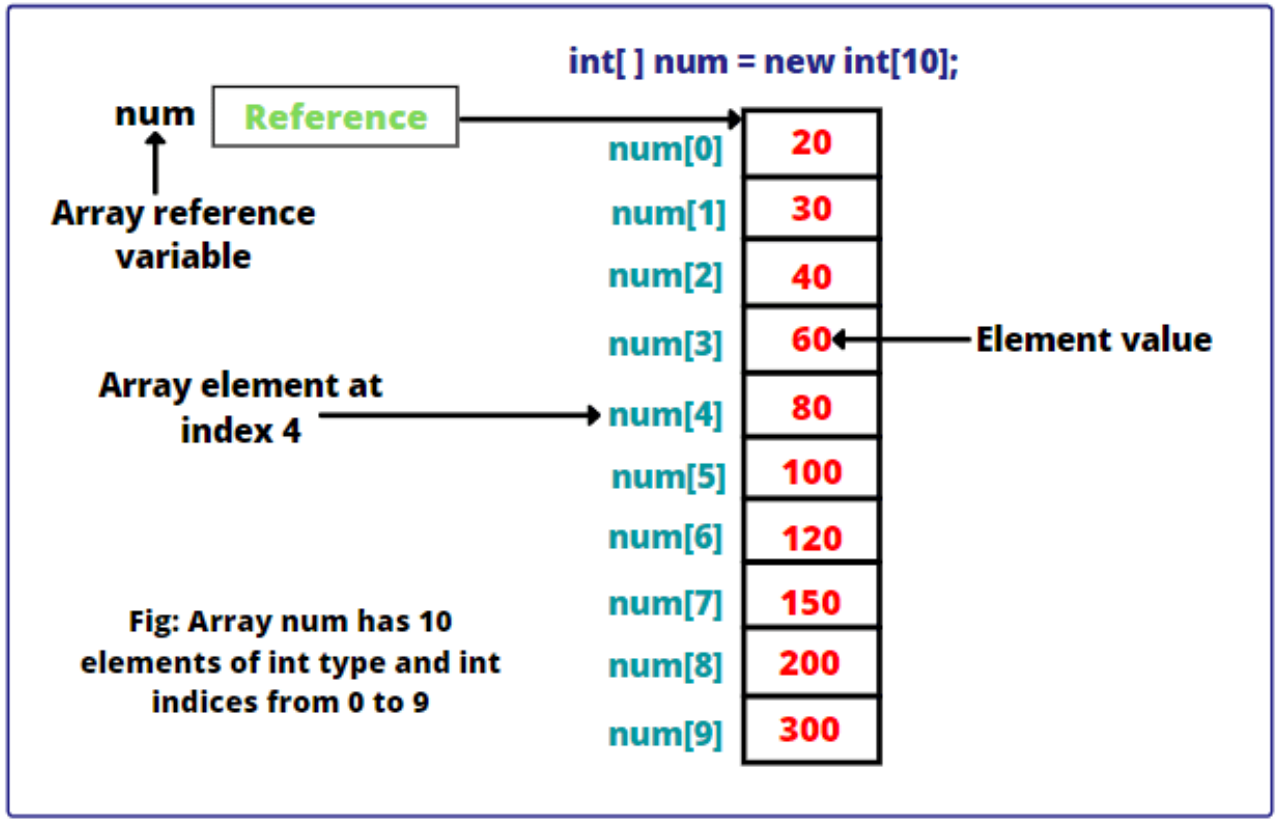


Fig: Array num has 10 elements of int type and int indices from 0 to 9

Arrays

Array: Finite, Ordered collection of Homogeneous data elements where,

Ordering is maintained by storing all elements in continuous / contiguous locations of memory.

Arrays

Properties:

Finite: Contains only a limited (finite) number of elements.

Ordered: All elements are stored one by one in continuous / contiguous locations of computer memory.

Homogeneous: All the elements of an array are of the same data type.

Example:

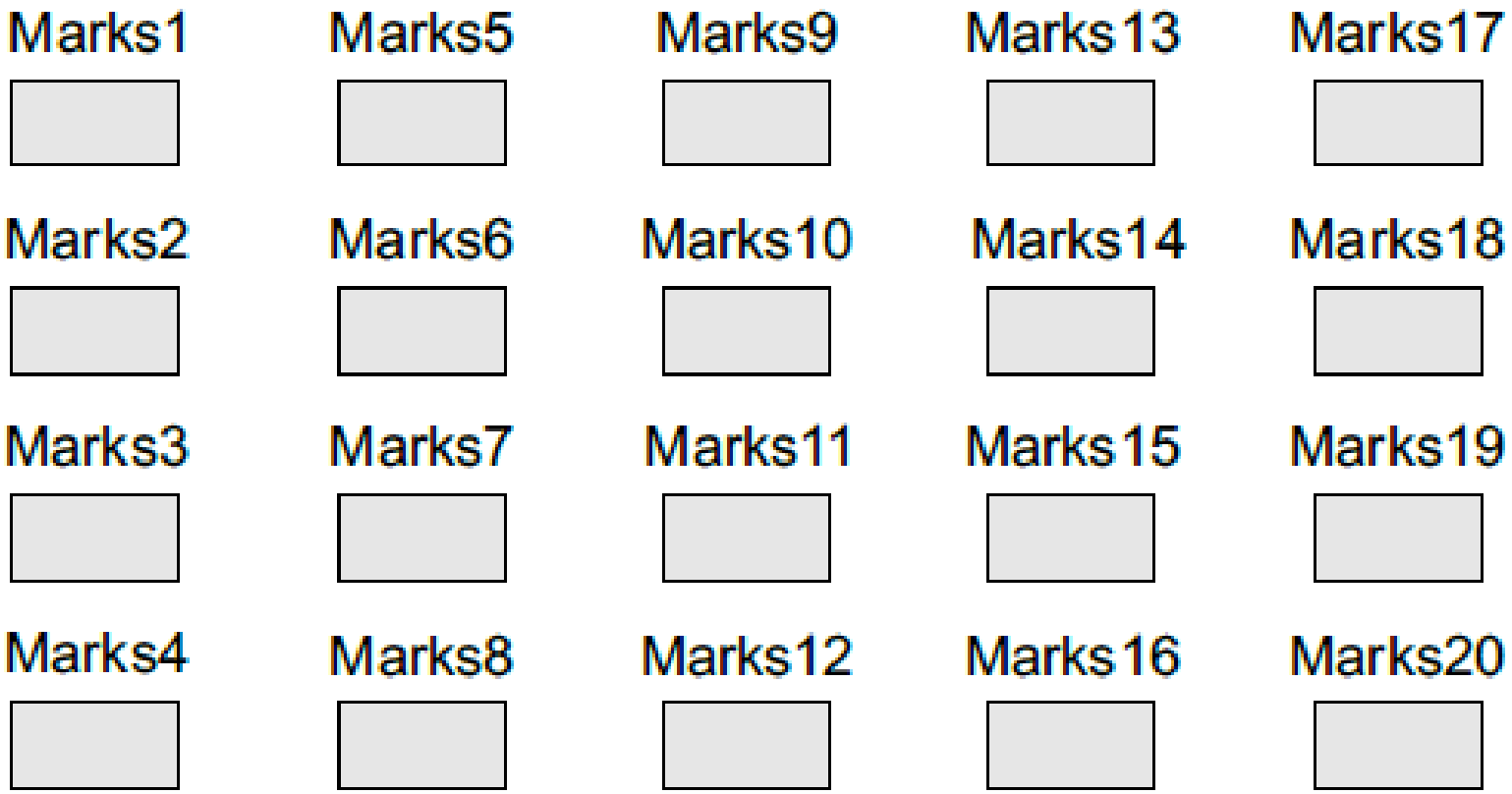
An array of integers to store the integer numbers of all students in a class.

An array of strings to store the names of students in a class.

Arrays

- Consider a situation in which we have 20 students in a class, and we have been asked to write a program that reads and prints the marks of all the 20 students.
- In this program, we will need 20 integer variables with different names, as shown in Fig. 1.1

Arrays -Fig. 1.1



Arrays

- If it is just a matter of 20 variables, then it might be acceptable for the user to follow this approach.
- But would it be possible to follow this approach if we have to read and print the marks of students,
- in the entire course (say 100 students)
- in the entire college (say 500 students)
- in the entire university (say 10,000 students)
- The answer is no, definitely not! To process a large amount of data,
- we need a data structure known as *array*.

Arrays

- An array is a collection of similar data elements.
- These data elements have the same data type.
- The elements of the array are stored in consecutive memory locations and are referenced by an index (also known as the *subscript*).
- The subscript is an ordinal number which is used to identify an element of the array

DECLARATION OF ARRAYS

- An array must be declared before being used.
- Declaring an array means specifying the following:
- ***Data type***—the kind of values it can store, for example, int, char, float, double.
- ***Name***—to identify the array.
- ***Size***—the maximum number of values that the array can hold.

E.g.

```
Int  num [ 10];
```

Figure 2.2: Memory representation of an array elements

1st element	2nd element	3rd element	4th element	5th element	6th element	7th element	8th element	9th element	10th element
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]	marks[5]	marks[6]	marks[7]	marks[8]	marks[9]

99	67	78	56	**88**	90	34	85
marks[0]	marks[1]	marks[2]	marks[3]	**marks[4]**	marks[5]	marks[6]	marks[7]
1000	1002	1004	1006	**1008**	1010	1012	1014

Basic Operations of arrays

Following are the basic operations supported by an array.

- **Traverse** – print/visit all the array elements one by one.
- **Insertion** – Adds an element at the given index/position .
- **Deletion** – Deletes an element at the given index/position.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Merging** :Merging two arrays
- **Sorting** : Sorting an array in ascending or descending order

ACCESSING THE ELEMENTS OF AN ARRAY

- Code 1 : initialization of the array.

```
// Set each element of the array to -1
int i, marks[10];
for(i=0;i<10;i++)
        marks[i] = -1;
```

Array ***marks*** after executing the code given in the above.

– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Traversing an Array

- Traversing an array means accessing each and every element of the array for a specific purpose.
- Traversing the data elements of an array A can include printing every element, counting the total number of elements, or performing any process on these elements.

Algorithm for array traversal

```
Step 1: [INITIALIZATION] SET I = lower_bound
Step 2: Repeat Steps 3 to 4 while I <= upper_bound
Step 3:        Apply Process to A[I]
Step 4:        SET  I = I + 1
        [END OF LOOP]
Step 5: EXIT
```

Traverse – Print all the elements in the array one by one

- Let's now see the pseudocode of this algorithm –

```
procedure print_array(A)
    FOR EACH value in A DO
    DISPLAY A[n]
    END FOR
end procedure
```

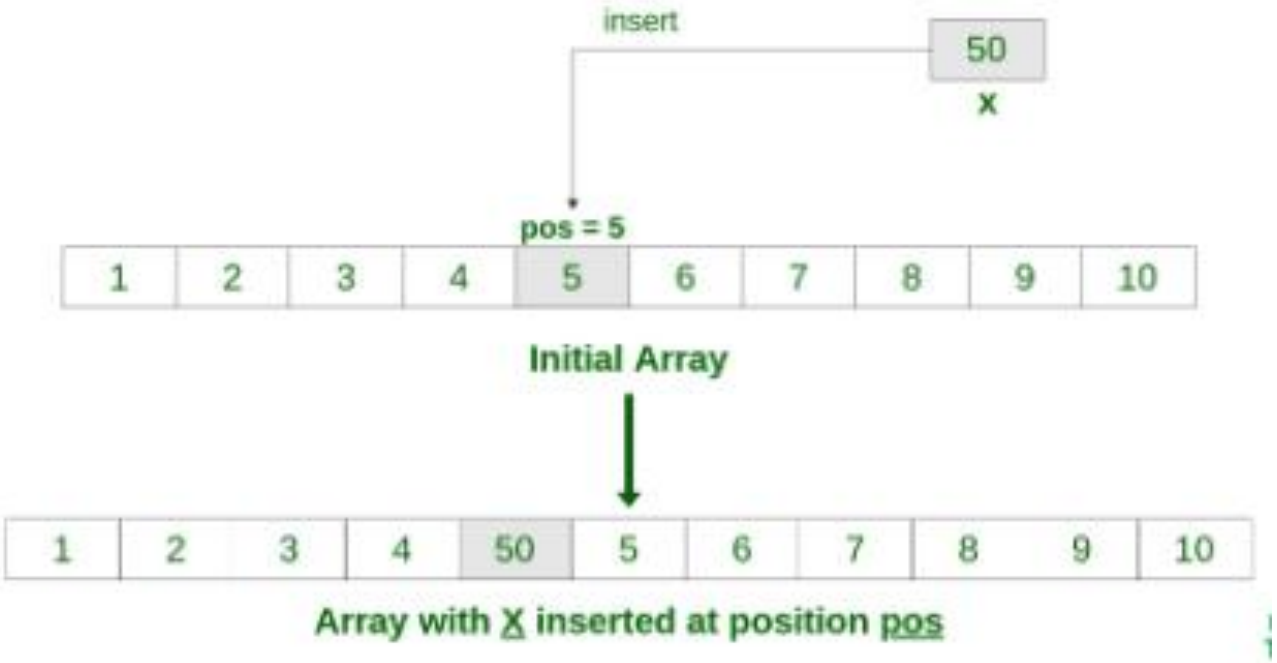
C Implementation

```c
#include <stdio.h>

int main() {
    int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
    int loop;

    for(loop = 0; loop < 10; loop++)
        printf("%d ", array[loop]);

    return 0;
}
```

OUTPUT :
1,2,3,4,5,6,7,8,9,0

How to Insert an element at a specific position

- Given an array **arr** of size **n**,
- insert an element **x** in this array **arr** at a specific position **pos**.



OUTPUT

```
Initial Array:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Array with 50 inserted at position 5:
[1, 2, 3, 4, 50, 5, 6, 7, 8, 9, 10]
```

How to Insert an element at a specific position

- First get the element to be inserted, say **x.**
- Then get the position at which this element is to be inserted, say **pos**
- Create a **new array** with the size one greater than the previous size.
- Copy all the elements from previous array into the new array till the position pos.
- Insert the element x at position pos.
- Insert the rest of the elements from the previous array into the new array after the pos.

Question 1

- Write a C program to insert an element at a specific position.

Searching an element in an array

- **Algorithm:**
- Step 1: Traverse the array
- Step 2: Match the key element with array element.
- Step 3: If key element is found, return the index position of the array element
- Step 4: If key element is not found, return -1

Given a array, Search a given element in array.

Case 1:

Input: Search 20.

12	5	10	15	31	20	25	2	40
0	1	2	3	4	5	6	7	8

Output: True (20 is present in array)

Case 2:

Input: Search 26.

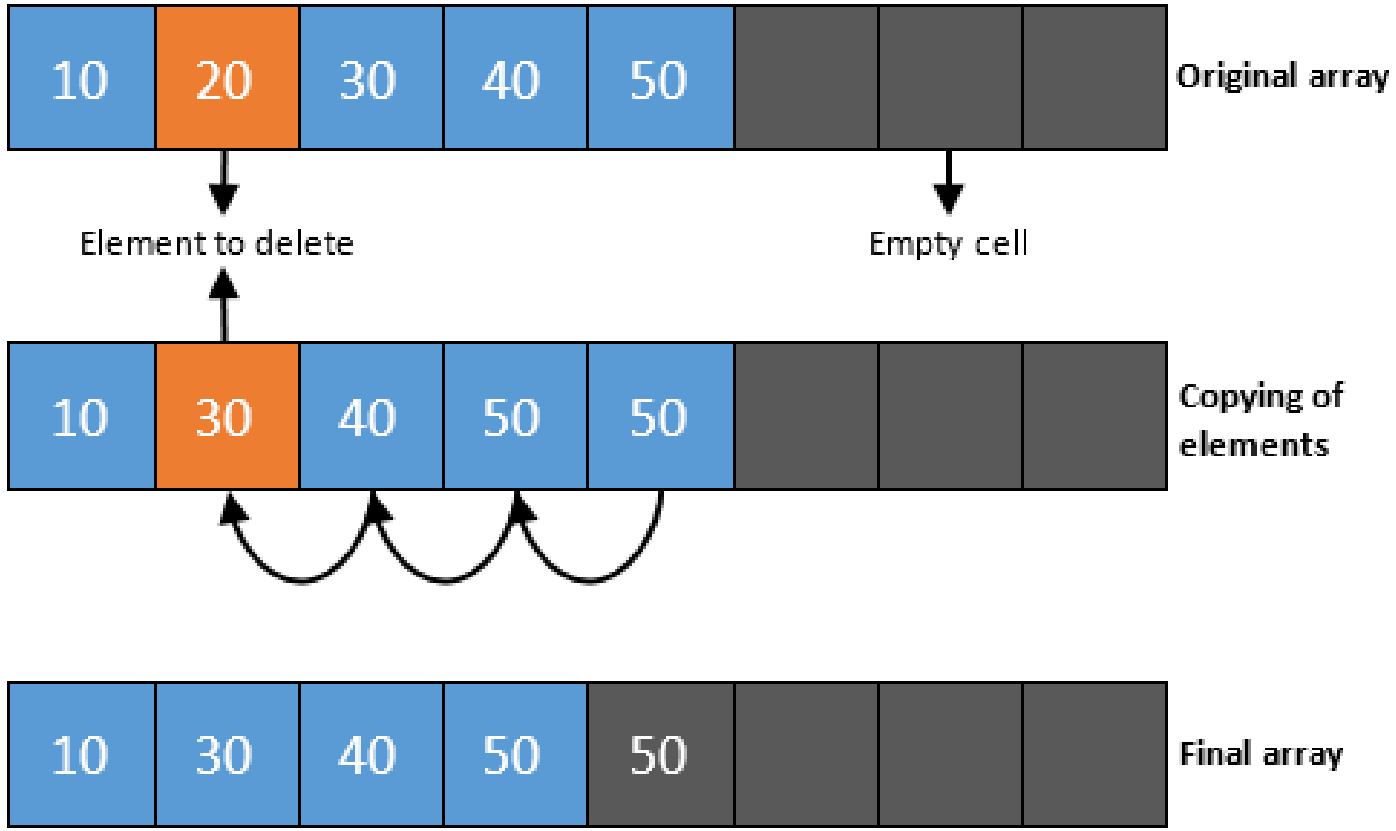
12	5	10	15	31	20	25	2	40
0	1	2	3	4	5	6	7	8

Output: False (26 is not present in array)

Question 2

- Write a C program to **search for an element in an array**.
  - The program should accept the size of the array and the array elements from the user.
  - Then, it should take an element to search for and display whether the element is **found** or **not found** in the array.

Deleting an element from an array

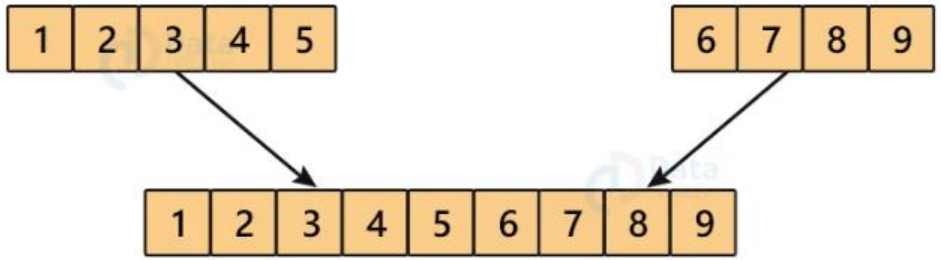


Question 3

- Write a C program to **delete an element from an array** at a given position.
  - The program should accept the size of the array and its elements from the user.
  - Then, it should take the position of the element to be deleted.
  - After deletion, display the new array without the deleted element.

Question 4

- Write an algorithm, **pseudo code**, and a **C program** to **merge two sorted arrays into one array**.
- **Requirements:**
- Accept the size and elements of two arrays from the user.
- Merge both arrays into a single array.
- Display the final merged array.
- E.g.



Homework 1.1

- Write a program to find the mean of *n* numbers using arrays.
- Write a program to print the position of the smallest number of *n* numbers using arrays.
- Write a program to find the second largest of *n* numbers using an array.
- Write a program to enter *n* number of digits. Form a number using these digits.

Question 5

- write a pseudocode algorithm to Increment all the array values by 1

Homework 1.2

- Write a program to find whether the array of integers contains a duplicate number.

Question 6

Data[] is an array that is declared as int Data[20]; and contains the following values:

Data[] = {12, 23, 34, 45, 56, 67, 78, 89, 90, 100};

a) Calculate the length of the array.

(b) Find the upper_bound and lower_bound.

(c) Show the memory representation of the array.

(d) If a new data element with the value 75 has to be inserted, find its position.

(e) Insert a new data element 75 and show the memory representation after the insertion.

Homework 1.3

- Write a program to insert a number in an array that is already sorted in ascending order.

Homework 1.3

- Write a program to delete a number from an array that is already sorted in ascending order.

Question 7

- Write an algorithm, pseudo code, and a C program to merge two sorted arrays into a single sorted array.
- **Requirements:**
  - Accept the size and elements of two arrays from the user.
  - Assume both arrays are already sorted in ascending order.
  - Merge them into a new array such that the final array is also sorted in ascending order.
  - Display the merged sorted array.
- Example

Array 1-	20	30	40	50	60							
Array 2-	15	22	31	45	56	62	78					
Array 3-	15	20	22	30	31	40	45	50	56	60	62	78

Homework 1.4

- Write a program to merge two arrays
- Extend the above program merge any number of arrays